

LLMs_Optimization

Милош Васић, [10. 11. 2025. 09:25] Is it possible to do distributed build - building AOSP codebase from main machine but with “attached” additional building workers - additional computers. If yes, I need step by step guide for this!

Милош Васић, [10. 11. 2025. 09:26] next:

Милош Васић, [10. 11. 2025. 09:26] According to the search results I obtained, distributed building for AOSP is technically possible, but a complete step-by-step guide is not available. The information is limited to mentions of the concept without detailed implementation instructions.

🤔 The Concept and Available Clues

The main tool mentioned for distributed compilation is distcc, which is designed to distribute C and C++ code compilation across multiple machines on a network. One user specifically asked about using it with the AOSP build process to overcome hardware limitations, confirming its theoretical applicability.

The modern AOSP build system relies on a toolchain that could, in principle, support distribution:

· Soong: The primary build system that processes Android.bp files. · Ninja: A lower-level build tool that executes the build plan and is highly optimized for speed and parallel execution. · Kati: A tool used to convert legacy Makefiles into Ninja files for backward compatibility.

Another potential, more modern avenue is Bazel. Google has been integrating Bazel into AOSP, particularly for kernel builds starting from newer Pixel devices. Bazel has built-in support for remote execution and caching, which is a form of distributed building. However, its support in AOSP is still evolving and is currently focused on specific components like the kernel rather than the entire OS.

💡 A Practical Path Forward and Key Considerations

Since the search results lack a concrete guide, here is a practical approach you can take to find the necessary information.

· Investigate distcc: Your search should focus on configuring distcc for large-scale projects. Key steps will involve: 1. Installing and configuring distcc on all machines (server and helpers). 2. Setting up the build environment on all helpers to ensure consistent toolchains. 3. Modifying the AOSP build command to use the distcc wrapper for the compiler. · Explore Bazel: For a more modern approach, look into the progress of Bazel integration within AOSP, particularly the “Kleaf” framework used for kernel builds.

Before you start, be aware of these significant challenges:

Challenge Description Configuration Complexity Synchronizing toolchains, include paths, and pre-built libraries across all machines is difficult. A single inconsistency can cause build failures. Network Performance The massive number of files in an AOSP build can make network latency and bandwidth a major bottleneck. Mixed Codebase AOSP contains not only C/C++ but also Java, which isn’t compiled with distcc. The overall time savings may be less than expected.

I hope this explanation helps you understand the landscape of distributed AOSP builds. Would you like me to elaborate more on the core components of the AOSP build system, such as Soong and Ninja, to help you better understand how a distributed process might work?

Милош Васић, [10. 11. 2025. 09:42] ****

Милош Васић, [10. 11. 2025. 09:42] When working with GLM-4.6 LLM model from Crush cli coding agent and project with big codebase, is it recommended to start a new session after each executed implementation request? It happens that everything slows down with very long pauses before anything “happening” and any work output produced as codebase size rises and we continue to give new requests. Any other particular advices maybe?

Милош Васић, [10. 11. 2025. 09:42] next:

Милош Васић, [10. 11. 2025. 09:42] Based on your description, the performance slowdown in Crush is a known challenge when working with large codebases, primarily due to the growing context size. While starting a new session can help, it’s not the only or always the best strategy.

Understanding the Slowdown and the “New Session” Trade-off

The GLM-4.6 model has a context window of 200K tokens. As your session continues, the conversation history and the provided context from your large codebase accumulate. The model has to process all this information for each new request, which naturally leads to longer processing times and the “long pauses” you’re experiencing.

Starting a new session clears this history, which can significantly speed up the model’s response time for the next task. However, this comes at a major cost: you lose all the context and “memory” of what was discussed and built in the previous session. For a complex, multi-step implementation, this loss of context can be detrimental.

Therefore, a new session is a good reset button if you are starting a completely unrelated task and speed is the absolute priority. For ongoing work on the same feature, the strategies below are more effective.

Strategies for Better Performance with Large Codebases

Instead of relying solely on session resets, you can optimize how you provide context to the model. The key is to be selective and precise to avoid overloading the session.

Strategy Core Principle Practical Application  Iterative Task Breakdown Break complex requests into smaller, focused steps. Instead of “add user authentication,” request “create a user schema,” then “write a registration function,” etc.  Context Precision Provide only the code and context essential for the immediate task. Use the @ command in Crush to include only specific, relevant files instead of the entire project structure.  Clear Specifications Use examples to define requirements precisely, reducing ambiguity and back-and-forth. Provide example inputs/outputs, coding standards, and error-handling requirements in your prompt.  Proactive Context Management Manually clear the chat history once a sub-task is complete and its context is no longer needed. After a function is successfully written and integrated, you can start a new session or simply remove the creation history.

Technical Configuration Check

It's also worth verifying your technical setup to ensure optimal performance.

1. Verify Your API Endpoint: For the best coding performance with Crush, ensure you are using the dedicated GLM Coding Plan endpoint: <https://api.z.ai/api/coding/paas/v4>[citation:1]. Using the common API might not be optimized for this workload.
2. Check Model Parameters: The official recommended parameters for coding tasks with GLM models are a temperature of 1.0, top_p of 0.95, and top_k of 40. Crush likely handles this, but it's a good setting to be aware of.

Hopefully, these strategies help you strike a better balance between performance and contextual awareness. If you continue to experience issues, you might want to check if there are any specific memory or resource limitations on your local machine while running Crush.

Милош Васић, [10. 11. 2025. 09:42] *****

Милош Васић, [10. 11. 2025. 10:27] GLM-4.6 optimization:

Милош Васић, [10. 11. 2025. 10:27] Unfortunately, you cannot make GLM-4.6 behave exactly like those other models because the performance differences stem from fundamental architectural differences that cannot be replicated through configuration alone. However, you can optimize your workflow to get the best possible performance from GLM-4.6.

What You Cannot Change

The key limitations are hardcoded into GLM-4.6's design:

- Fixed Architecture: GLM-4.6 uses a dense transformer architecture, not the more efficient Mixture of Experts (MoE) that models like DeepSeek V3.1 use
- Context Processing: The way it handles long contexts is baked into the model weights and architecture
- No "Thinking" Mode: Unlike DeepSeek, GLM-4.6 doesn't have a separate reasoning mode you can activate

What You CAN Do to Optimize GLM-4.6

Here are practical strategies to maximize GLM-4.6's performance with large codebases:

1. Aggressive Context Management

Be extremely selective about what files you include

@src/core/feature.ts # Instead of @src/ @src/utils/helpers.ts 2. Implement Manual "Checkpointing"

- After completing a significant feature, save the key decisions and current state in a text file
- Start a new session and provide that summary file as context
- This simulates the task coherence that Claude models maintain naturally

1. Structured Prompt Engineering

Instead of:

"Add authentication to the user service" Use:

“Step 1: Create a User model with email and hashed_password fields Step 2: Implement bcrypt password hashing in auth_utils.py Step 3: Add login endpoint to user_routes.py Current relevant files: @models/user.py @routes/base.py” 4. External Tool Integration

Since GLM-4.6 can't natively maintain context like the other models, use external tools:

- Code search tools (ripgrep, silver searcher) to quickly find relevant code
- Documentation generators to maintain project context
- Session logging to preserve important decisions between sessions

1. Parameter Optimization

While Crush may have defaults, you could experiment with:

–temperature 0.1 # More deterministic for code –top-p 0.9 –max-tokens 4096 # Balance between completeness and speed 🌀 Recommended Workflow Adaptation

Given GLM-4.6's limitations, I'd suggest this hybrid approach:

1. Use GLM-4.6 for focused, context-limited tasks where you can provide just 2-3 relevant files
2. Start fresh sessions frequently - accept this as necessary with this model
3. Maintain external documentation of architectural decisions
4. Consider using multiple models - use GLM-4.6 for quick tasks and switch to one of the more capable models for complex, multi-session work

🤔 Reality Check

The performance gap you're experiencing is exactly why models like Claude Sonnet 4.5 and DeepSeek V3.1 were specifically engineered for agentic coding workflows. They're fundamentally built differently for this use case.

If your projects consistently involve large codebases and complex multi-step tasks, you might find that switching to one of those purpose-built models ultimately saves more time than trying to optimize GLM-4.6 beyond its architectural limits.

Would you like me to suggest specific strategies for integrating multiple models into your workflow, or would you prefer to focus on maximizing what you can achieve with GLM-4.6 given these constraints?

Милош Васић, [10. 11. 2025. 10:27] next:

Милош Васић, [10. 11. 2025. 10:32] What would happen if all those 'optimizations' we apply to all other models, even to mentioned 'strong' ones? Will it be beneficial for them too?

Милош Васић, [10. 11. 2025. 10:32] Excellent question! Yes, absolutely - applying those optimizations to the “strong” models would be highly beneficial and would actually make them perform even better. The key insight is that these optimizations are fundamentally about working efficiently with LLMs, not just compensating for weaknesses.

📈 Why Strong Models Benefit MORE from Optimizations

Optimization	Effect on Weaker Models	Effect on Strong Models
Context Precision	Prevents overload/confusion	Allows focusing superior reasoning on what matters
Iterative Breakdown	Makes tasks achievable	Enables tackling even more complex problems
Clear Specifications		

Reduces basic errors Leverages advanced reasoning for elegant solutions External Tool Integration Compensates for limitations Amplifies already strong capabilities

Strong models have more sophisticated reasoning capabilities, so when you give them clean, well-structured inputs, they can leverage their advanced abilities much more effectively.

The Multiplier Effect

Think of it this way:

· GLM-4.6 with optimizations: 60% → 80% of potential · Claude Sonnet 4.5 with optimizations: 85% → 95%+ of potential

The strong models actually get more relative benefit because they're not fighting against architectural limitations while trying to process your optimized inputs.

Advanced Optimizations That Unlock Superpowers

With the top-tier models, you can implement even more sophisticated optimizations:

1. Progressive Context Loading

Instead of dumping all context at once

```
def build_context(task_type, current_focus):  
    base_context = load_architecture_overview()  
    if task_type == "api":  
        base_context += load_related_routes_and_models()  
    if current_focus == "auth":  
        base_context += load_auth_specific_files()  
    return base_context
```

2. Meta-Cognitive Prompting

“Before implementing, please: 1. Analyze the current code structure patterns 2. Identify consistent naming conventions 3. Note any established error handling approaches 4. Then proceed with the implementation”

3. Cross-File Dependency Mapping

Strong models can handle complex dependency graphs and actually suggest better architecture based on seeing patterns across your entire codebase.

The Real Difference: Optimization Ceiling

The fundamental difference isn't whether optimizations help, but how high the performance ceiling goes when you apply them:

· GLM-4.6: Optimizations prevent degradation and maintain basic functionality · Strong models: Optimizations unlock superlinear performance gains - the model starts suggesting architectural improvements, identifying hidden bugs, and proposing optimizations you hadn't considered

Practical Example: Database Migration

With GLM-4.6 + Optimizations:

· Successfully creates migration files · Follows existing patterns · Gets the job done without breaking things

With Claude Sonnet 4.5 + Same Optimizations:

· Creates migration files · Notices inconsistent indexing patterns across similar tables · Suggests a more efficient schema design · Identifies potential data integrity issues in the migration path · Proposes a rollback strategy

The Bottom Line

Yes, you should apply ALL these optimizations to every model you use. The ROI is actually higher with stronger models because:

1. They leverage good inputs more effectively
2. They can handle more sophisticated optimization strategies
3. The performance gains compound due to their advanced reasoning
4. You get additional “emergent benefits” like architectural insights

The optimizations aren’t crutches for weak models - they’re force multipliers that work across the entire spectrum, but particularly amplify already powerful capabilities.

Would you like me to elaborate on any of these advanced optimization strategies for the specific strong models you’re considering?

Милош Васић, [10. 11. 2025. 10:33] next:

Милош Васић, [10. 11. 2025. 10:33] Yes give me as much as possible fine tuning and implementation steps in order for this to work flawlessly with any LLM - api and local ones both!

Милош Васић, [10. 11. 2025. 10:33] next:

Милош Васић, [10. 11. 2025. 10:39]  Universal LLM Optimization Framework

Complete implementation guide for flawless performance with any LLM

Here’s your comprehensive, step-by-step framework that works across all LLMs (API and local).

Phase 1: Foundation Setup

1.1 Context Management System

context_manager.py

```
import os
import ast
from pathlib import Path
from typing import Dict, List, Set
```

```
class ContextManager:
    def __init__(self, project_root: str, max_context_files: int = 10):
        self.project_root = Path(project_root)
        self.max_context_files = max_context_files
        self.dependency_graph = {}
```

```
    def get_relevant_files(self, task_description: str, focus_areas: List[str]) -> Set[str]:
        """Intelligently select files based on task and focus areas"""
        relevant = set()
```

```
        # 1. File content analysis
```

```
        for file_path in self.project_root.rglob("*.py"):
```

```
            if self._is_file_relevant(file_path, task_description, focus_areas):
                relevant.add(str(file_path))
```

```

# 2. Import dependency analysis
relevant.update(self._get_import_dependencies(relevant))

# 3. Limit and prioritize
return self._prioritize_files(list(relevant))[:self.max_context_files]

def _is_file_relevant(self, file_path: Path, task: str, focus_areas: List[str]):
    """Check if file content matches task keywords"""
    try:
        content = file_path.read_text()
        keywords = self._extract_keywords(task) + focus_areas

        # Check for keyword matches in classes/functions
        tree = ast.parse(content)
        for node in ast.walk(tree):
            if isinstance(node, (ast.ClassDef, ast.FunctionDef)):
                if any(kw.lower() in node.name.lower() for kw in keywords):
                    return True

        # Check imports and comments
        if any(kw in content for kw in keywords):
            return True

    except:
        pass
    return False

```

1.2 Project Analysis & Mapping

project_analyzer.py

```

import subprocess
import json
from collections import defaultdict

class ProjectAnalyzer:
    def __init__(self, project_root: str):
        self.project_root = project_root

    def create_architecture_map(self) -> Dict:
        """Generate comprehensive project structure analysis"""
        return {
            "entry_points": self._find_entry_points(),
            "dependencies": self._analyze_dependencies(),
            "key_patterns": self._identify_patterns(),
            "recent_changes": self._get_recent_changes()
        }

    def _find_entry_points(self) -> List[str]:
        """Identify main application entry points"""
        entry_files = []
        for pattern in ["main.py", "app.py", "index.*", "server.py"]:
            entry_files.extend(self.project_root.glob(pattern))
        return [str(f) for f in entry_files]

    def _analyze_dependencies(self) -> Dict[str, List[str]]:
        """Map file dependencies using AST analysis"""

```

```

dependencies = defaultdict(list)
for py_file in self.project_root.rglob("*.py"):
    try:
        content = py_file.read_text()
        tree = ast.parse(content)

        for node in ast.walk(tree):
            if isinstance(node, ast.Import):
                for alias in node.names:
                    dependencies[str(py_file)].append(alias.name)
            elif isinstance(node, ast.ImportFrom):
                if node.module:
                    dependencies[str(py_file)].append(node.module)
    except:
        continue
return dict(dependencies)

```

🌀 Phase 2: Prompt Engineering System

2.1 Structured Prompt Builder

`python # prompt_engineer.py from datetime import datetime from typing import Dict, List

Милош Васић, [10. 11. 2025. 10:39] class PromptEngineer: def init(self, context_manager: ContextManager, project_analyzer: ProjectAnalyzer): self.context_manager = context_manager self.project_analyzer = project_analyzer

```

def build_task_prompt(self, task: str, task_type: str, complexity: str = "
    """Build optimized prompt for any task type"""
    base_template = self._get_template(task_type)
    context_files = self.context_manager.get_relevant_files(
        task, self._extract_focus_areas(task)
    )

```

```

    prompt = base_template.format(
        task=task,
        context=self._build_context_string(context_files),
        constraints=self._get_constraints(complexity),
        examples=self._get_examples(task_type),
        timestamp=datetime.now().isoformat()
    )

```

```

    return {
        "prompt": prompt,
        "metadata": {
            "task_type": task_type,
            "complexity": complexity,
            "context_files": context_files,
            "token_estimate": len(prompt) // 4
        }
    }

```

def _get_template(self, task_type: str) -> str:


```
templates = {  
    "implementation": """\
```

TASK: {task}

TIMESTAMP: {timestamp}

CONTEXT:

{context}

CONSTRAINTS:

{constraints}

IMPLEMENTATION REQUIREMENTS:

1. Follow existing code patterns and style
2. Maintain consistency with architecture
3. Include appropriate error handling
4. Add necessary tests if applicable
5. Document complex logic

EXPECTED OUTPUT:

- Provide complete, runnable code
- Include necessary imports
- Follow project conventions

YOUR RESPONSE:““““,

```
    "refactor": """\
```

REFACTOR TASK: {task}

TIMESTAMP: {timestamp}

CURRENT CODE:

{context}

REFACTORING GOALS:

{constraints}

REFACTORING PRINCIPLES:

1. Don't break existing functionality
2. Improve readability/maintainability
3. Follow SOLID principles where applicable
4. Maintain or improve performance

EXPECTED OUTPUT:

- Show before/after if helpful
- Explain key changes
- Ensure backward compatibility""" } return templates.get(task_type, templates["implementation"])

2.2 Iterative Task Decomposer

```
python # task_decomposer.py class TaskDecomposer: def init(self, llm_client): self.llm_client = llm_client
```

```
def decompose_complex_task(self, complex_task: str) -> List[Dict]:
    """Break complex tasks into executable steps"""
    decomposition_prompt = f"""
    Analyze this complex development task and break it into sequential, te

    TASK: {complex_task}

    For each step, provide:
    1. Step description
    2. Input files needed
    3. Expected output
    4. Validation criteria
    5. Estimated effort (S/M/L)

    Format as JSON:
    {{
        "steps": [
            {{
                "step": 1,
                "description": "...",
                "inputs": ["file1", "file2"],
                "outputs": ["file1_modified", "new_file"],
                "validation": "how to verify step success",
                "effort": "S"
            }}
        ]
    }}
    """
```

```
response = self.llm_client.complete(decomposition_prompt)
return self._parse_decomposition(response)
```

```
def create_implementation_plan(self, steps: List[Dict]) -> str:
    """Convert steps into executable plan"""
```

```

plan = ["# IMPLEMENTATION PLAN"]
for step in steps:
    plan.append(f"\n## Step {step['step']}: {step['description']}")
    plan.append(f"Inputs: {' ', ' '.join(step['inputs'])}")
    plan.append(f"Outputs: {' ', ' '.join(step['outputs'])}")
    plan.append(f"Validation: {step['validation']}")
    plan.append(f"Effort: {step['effort']}")

return "\n".join(plan)

```

Phase 3: Execution & Validation System

3.1 Multi-Model Client Interface

Милош Васић, [10. 11. 2025. 10:39] # llm_client.py import openai import requests from typing import Dict, Any

```

class UniversalLLMClient:
    def __init__(self, config: Dict[str, Any]):
        self.config = config
        self.supported_models = self._init_clients()

    def complete(self, prompt: str, model: str = None, **kwargs) -> str:
        """Unified completion interface for all LLMs"""
        model = model or self.config["default_model"]
        client_config = self.supported_models[model]

        if client_config["type"] == "openai":
            return self._openai_complete(prompt, model, **kwargs)
        elif client_config["type"] == "anthropic":
            return self._anthropic_complete(prompt, model, **kwargs)
        elif client_config["type"] == "local":
            return self._local_complete(prompt, model, **kwargs)
        elif client_config["type"] == "custom_api":
            return self._custom_api_complete(prompt, model, **kwargs)

    def _openai_complete(self, prompt: str, model: str, **kwargs) -> str:
        """Handle OpenAI-compatible APIs"""
        client = openai.OpenAI(api_key=self.config["api_keys"]["openai"])
        response = client.chat.completions.create(
            model=model,
            messages=[{"role": "user", "content": prompt}],
            temperature=kwargs.get("temperature", 0.1),
            max_tokens=kwargs.get("max_tokens", 4000)
        )
        return response.choices[0].message.content

    def _local_complete(self, prompt: str, model: str, **kwargs) -> str:
        """Handle local models via Ollama or similar"""
        # Example for Ollama
        response = requests.post(
            "http://localhost:11434/api/generate",
            json={
                "model": model,

```

```

        "prompt": prompt,
        "stream": False,
        "options": {
            "temperature": kwargs.get("temperature", 0.1),
            "num_predict": kwargs.get("max_tokens", 4000)
        }
    )
    return response.json()["response"]

```

3.2 Code Validation & Integration

code_validator.py

```
import subprocess
import tempfile
import ast
from typing import Tuple, List
```

```
class CodeValidator:
    def __init__(self, project_root: str):
        self.project_root = project_root
```

```

    def validate_and_integrate(self, generated_code: str, target_file: str) -> str:
        """Validate generated code and prepare for integration"""
        validation_steps = [
            self._syntax_validation,
            self._import_validation,
            self._style_consistency,
            self._test_integration
        ]

```

```

        issues = []
        for validator in validation_steps:
            valid, message = validator(generated_code, target_file)
            if not valid:
                issues.append(message)

```

```
        return len(issues) == 0, "\n".join(issues)
```

```

    def _syntax_validation(self, code: str, target_file: str) -> Tuple[bool, str]:
        """Validate Python syntax"""
        try:
            ast.parse(code)
            return True, "Syntax OK"
        except SyntaxError as e:
            return False, f"Syntax error: {e}"

```

```

    def _create_safe_test_environment(self, code: str) -> str:
        """Create isolated environment for testing generated code"""
        with tempfile.NamedTemporaryFile(mode='w', suffix='.py', delete=False):
            f.write("""

```

```
import sys
import os
sys.path.insert(0, os.getcwd())
```

```

try:
    f.write(code)
    f.write("""
print("EXECUTION_SUCCESS")
except Exception as e:
    print(f"EXECUTION_ERROR: {e}")
""")
    return f.name

```



Phase 4: Advanced Optimization System

4.1 Performance Monitoring & Adaptive Tuning

```
`python # performance_monitor.py import time import psutil from dataclasses import dataclass
from typing import Dict, List import statistics
```

Милош Васић, [10. 11. 2025. 10:39] @dataclass class PerformanceMetrics: response_time: float
token_throughput: float context_efficiency: float success_rate: float

```
class PerformanceMonitor: def init(self): self.metrics_history = [] self.model_performance = {}
```

```
def record_interaction(self, model: str, prompt_tokens: int,
                        response_tokens: int, response_time: float,
                        success: bool):
```

```
    """Record performance metrics for each interaction"""
```

```
    metrics = PerformanceMetrics(
        response_time=response_time,
        token_throughput=response_tokens / response_time if response_time
        context_efficiency=response_tokens / prompt_tokens if prompt_tokens
        success_rate=1.0 if success else 0.0
    )
```

```
    if model not in self.model_performance:
        self.model_performance[model] = []
    self.model_performance[model].append(metrics)
```

```
def get_optimal_parameters(self, model: str, task_type: str) -> Dict:
    """Recommend optimal parameters based on historical performance"""
    if model not in self.model_performance:
        return self._get_default_parameters(task_type)
```

```
    model_metrics = self.model_performance[model]
    recent_metrics = model_metrics[-10:] # Last 10 interactions
```

```
    # Adaptive tuning based on performance
    avg_throughput = statistics.mean(m.token_throughput for m in recent_metrics)
    avg_success = statistics.mean(m.success_rate for m in recent_metrics)
```

```
    base_params = self._get_default_parameters(task_type)
```

```
    # Adjust based on performance
    if avg_throughput < 50: # tokens per second
        base_params["max_tokens"] = min(base_params["max_tokens"], 2000)
    if avg_success > 0.8:
        base_params["temperature"] = min(base_params["temperature"] + 0.1,
```

```
        return base_params
```

4.2 Cross-Model Session Management

```
python # session_manager.py import json import hashlib from datetime import datetime,
timedelta
```

```
class CrossModelSessionManager: def init(self, session_timeout_minutes: int = 60): self.sessions
= {} self.session_timeout = timedelta(minutes=session_timeout_minutes)
```

```

def create_session(self, project_id: str, initial_context: Dict) -> str:
    """Create new session with context"""
    session_id = hashlib.md5(f"{project_id}{datetime.now()}".encode()).hexdigest()

    self.sessions[session_id] = {
        "project_id": project_id,
        "created_at": datetime.now(),
        "context": initial_context,
        "interactions": [],
        "model_usage": {},
        "checkpoints": []
    }
    return session_id

def update_session(self, session_id: str, interaction: Dict, model: str):
    """Update session with new interaction"""
    if session_id in self.sessions:
        session = self.sessions[session_id]
        session["interactions"].append(interaction)
        session["model_usage"][model] = session["model_usage"].get(model, 0) + 1

        # Auto-create checkpoint every 5 interactions
        if len(session["interactions"]) % 5 == 0:
            self._create_checkpoint(session_id)

def get_session_summary(self, session_id: str) -> str:
    """Generate session summary for context carryover"""
    session = self.sessions.get(session_id, {})
    interactions = session.get("interactions", [])

    summary = [f"# SESSION SUMMARY - {session_id}"]
    summary.append(f"Project: {session.get('project_id', 'Unknown')}")
    summary.append(f"Interactions: {len(interactions)}")

```

Милош Васић, [10. 11. 2025. 10:39] summary.append("## Key Decisions:")

```

# Extract key implementation decisions
for i, interaction in enumerate(interactions[-5:], 1): # Last 5
    if "decision" in interaction.get("metadata", {}):
        summary.append(f"{i}. {interaction['metadata']['decision']}")

return "\n".join(summary)

```

Phase 5: Integration & Workflow

5.1 Main Orchestrator Class

```

python # llm_orchestrator.py class LLMOrchestrator:
def init(self, project_root: str, config: Dict):
    self.project_root = project_root
    self.config = config

    # Initialize all components
    self.context_manager = ContextManager(project_root)
    self.project_analyzer = ProjectAnalyzer(project_root)
    self.prompt_engineer = PromptEngineer(self.context_manager, self.project_analyzer)

```

```

self.llm_client = UniversalLLMClient(config)
self.task_decomposer = TaskDecomposer(self.llm_client)
self.validator = CodeValidator(project_root)
self.performance_monitor = PerformanceMonitor()
self.session_manager = CrossModelSessionManager()

self.current_session = None

def execute_complex_task(self, task: str, model: str = None) -> Dict:
    """Main method to execute any complex task"""

    # 1. Start or resume session
    if not self.current_session:
        arch_map = self.project_analyzer.create_architecture_map()
        self.current_session = self.session_manager.create_session(
            self.project_root, arch_map
        )

    # 2. Decompose task if complex
    if self._is_complex_task(task):
        steps = self.task_decomposer.decompose_complex_task(task)
        plan = self.task_decomposer.create_implementation_plan(steps)

        results = []
        for step in steps:
            result = self._execute_single_step(step, model)
            results.append(result)

        return {"type": "complex", "steps": results, "plan": plan}
    else:
        # 3. Execute single task
        result = self._execute_single_task(task, model)
        return {"type": "simple", "result": result}

def _execute_single_step(self, step: Dict, model: str) -> Dict:
    """Execute a single step from decomposition"""
    prompt_data = self.prompt_engineer.build_task_prompt(
        step["description"], "implementation", step["effort"]
    )

    # Get optimal parameters for this model
    optimal_params = self.performance_monitor.get_optimal_parameters(
        model, "implementation"
    )

    start_time = time.time()
    response = self.llm_client.complete(
        prompt_data["prompt"], model, **optimal_params
    )
    response_time = time.time() - start_time

    # Validate and integrate
    success, validation_msg = self.validator.validate_and_integrate(
        response, step["outputs"][0] if step["outputs"] else "unknown.py"

```

```

)

# Record performance
self.performance_monitor.record_interaction(
    model, len(prompt_data["prompt"]), len(response),
    response_time, success
)

# Update session
self.session_manager.update_session(
    self.current_session,
    {"step": step, "response": response, "success": success},
    model
)

return {
    "step": step["description"],
    "response": response,
    "success": success,
    "validation": validation_msg,
    "performance": {
        "response_time": response_time,
        "tokens_used": len(response)
    }
}

```

5.2 Configuration Template

Милош Васић, [10. 11. 2025. 10:39] # config_template.py

```

DEFAULT_CONFIG = {
    "default_model": "gpt-4", "fallback_models": ["claude-3-sonnet", "local-codellama"],
    "api_keys": { "openai": "your-key", "anthropic": "your-key", "groq": "your-key" },
    "local_models": { "codellama": "codellama:13b-instruct", "deepseek-coder": "deepseek-coder:latest" },
    "optimization": { "max_context_files": 8, "auto_session_restart": True,
    "performance_monitoring": True, "adaptive_parameters": True },
    "parameters": {
        "implementation": { "temperature": 0.1, "max_tokens": 4000 },
        "refactor": { "temperature": 0.2, "max_tokens": 3000 },
        "debug": { "temperature": 0.05, "max_tokens": 2000 },
        "documentation": { "temperature": 0.3, "max_tokens": 2500 } } }

```

🚀 Quick Start Implementation

quick_start.py

```

def main(): # 1. Initialize orchestrator
    orchestrator = LLMOrchestrator( project_root="/path/to/your/project", config=DEFAULT_CONFIG )

    # 2. Execute complex task
    result = orchestrator.execute_complex_task(
        "Add user authentication system with JWT tokens",
        model="gpt-4"
    )

    # 3. Handle results
    if result["type"] == "complex":

```



```

for step_result in result["steps"]:
    if step_result["success"]:
        print(f"✅ {step_result['step']}")
    else:
        print(f"❌ {step_result['step']}: {step_result['validation']}")

```

```

# 4. Get session summary for next session
summary = orchestrator.session_manager.get_session_summary(
    orchestrator.current_session
)
print("Session summary for next session:")
print(summary)

```

if **name** == “**main**”: main() This framework provides:

· 🧩 Modular components you can use separately · 🔧 Universal compatibility with any LLM · 🇮🇹
 Performance optimization across all models · 🔄 Session management that works locally or via
 API · 🎯 Adaptive prompting for different task types · ✅ Validation & integration pipelines

Start with the quick start and gradually integrate components based on your specific needs!

Милош Васић, [10. 11. 2025. 10:39] *****